

NASM x86_64 – Mini-exemples pour pratiquer l’assembleur sous Linux

Bintou Flamouso Diallo

Septembre 2026

Contents

1 Exercice 1 – Hello World (syscall)	2
2 Exercice 2 – Exit code personnalisé	2
3 Exercice 3 – Afficher un entier avec printf	2
4 Exercice 4 – Lire un entier avec scanf	3
5 Exercice 5 – Addition simple	3
6 Exercice 6 – Comparer deux valeurs	3
7 Exercice 7 – Boucle simple (compter de 5 à 1)	4
8 Exercice 8 – Boucle avec condition (tant que > 0)	4
9 Exercice 9 – Manipuler un tableau d’entiers	4
10 Exercice 10 – Trouver le maximum dans un tableau	4
11 Exercice 11 – Implémenter strlen	5
12 Exercice 12 – Implémenter strcpy	5
13 Exercice 13 – Implémenter atoi	5
14 Exercice 14 – Mini-calculatrice (+, -, *, /)	6
15 Exercice 15 – Appeler une fonction ASM depuis C	6

Introduction

Ce document regroupe 15 mini-exemples en assembleur NASM x86_64 sous Linux. Chaque exercice possède :

- un dossier dédié dans le dépôt Git ;
- un fichier `.asm` contenant le code source ;
- un fichier `README.md` décrivant l'objectif, les commandes et l'output attendu.

1 Exercice 1 – Hello World (syscall)

Objectif

Afficher un message simple en utilisant le syscall `write`.

Code source

Chemin du fichier :

- `ex01_hello/ex01.asm`

Compilation et exécution

- `nasm -f elf64 ex01.asm -o ex01.o`
- `ld ex01.o -o ex01`
- `./ex01`

2 Exercice 2 – Exit code personnalisé

Objectif

Utiliser le syscall `exit` pour retourner un code de sortie spécifique.

Code source

- `ex02_exitcode/ex02.asm`

Compilation et exécution

- `nasm -f elf64 ex02.asm -o ex02.o`
- `ld ex02.o -o ex02`
- `./ex02`

3 Exercice 3 – Afficher un entier avec printf

Objectif

Appeler `printf` pour afficher un entier en utilisant la convention d'appel System V.

Code source

- `ex03_printf/ex03.asm`

Compilation et exécution

- `nasm -f elf64 ex03.asm -o ex03.o`
- `gcc ex03.o -o ex03`
- `./ex03`

4 Exercice 4 – Lire un entier avec scanf

Objectif

Lire un entier depuis l'entrée standard avec `scanf` et l'afficher.

Code source

- `ex04_scanf/ex04.asm`

Compilation et exécution

- `nasm -f elf64 ex04.asm -o ex04.o`
- `gcc ex04.o -o ex04`
- `./ex04`

5 Exercice 5 – Addition simple

Objectif

Additionner deux entiers et afficher le résultat.

Code source

- `ex05_add/ex05.asm`

Compilation et exécution

- `nasm -f elf64 ex05.asm -o ex05.o`
- `gcc ex05.o -o ex05`
- `./ex05`

6 Exercice 6 – Comparer deux valeurs

Objectif

Comparer deux entiers et afficher un message selon le résultat (<, =, >).

Code source

- `ex06_compare/ex06.asm`

Compilation et exécution

- `nasm -f elf64 ex06.asm -o ex06.o`
- `gcc ex06.o -o ex06`
- `./ex06`

7 Exercice 7 – Boucle simple (compter de 5 à 1)

Objectif

Utiliser une boucle avec `dec` et `jmp` pour compter de 5 à 1.

Code source

- `ex07_loop/ex07.asm`

Compilation et exécution

- `nasm -f elf64 ex07.asm -o ex07.o`
- `gcc ex07.o -o ex07`
- `./ex07`

8 Exercice 8 – Boucle avec condition (tant que > 0)

Objectif

Implémenter une boucle qui décrémente une valeur tant qu'elle est strictement positive.

Code source

- `ex08_while/ex08.asm`

Compilation et exécution

- `nasm -f elf64 ex08.asm -o ex08.o`
- `gcc ex08.o -o ex08`
- `./ex08`

9 Exercice 9 – Manipuler un tableau d'entiers

Objectif

Parcourir un tableau d'entiers en mémoire et afficher chaque élément.

Code source

- `ex09_array/ex09.asm`

Compilation et exécution

- `nasm -f elf64 ex09.asm -o ex09.o`
- `gcc ex09.o -o ex09`
- `./ex09`

10 Exercice 10 – Trouver le maximum dans un tableau

Objectif

Parcourir un tableau d'entiers et déterminer la valeur maximale.

Code source

- `ex10_max/ex10.asm`

Compilation et exécution

- `nasm -f elf64 ex10.asm -o ex10.o`
- `gcc ex10.o -o ex10`
- `./ex10`

11 Exercice 11 – Implémenter `strlen`

Objectif

Compter le nombre de caractères dans une chaîne terminée par 0.

Code source

- `ex11_strlen/ex11.asm`

Compilation et exécution

- `nasm -f elf64 ex11.asm -o ex11.o`
- `gcc ex11.o -o ex11`
- `./ex11`

12 Exercice 12 – Implémenter `strcpy`

Objectif

Copier une chaîne source vers une chaîne destination caractère par caractère.

Code source

- `ex12_strcpy/ex12.asm`

Compilation et exécution

- `nasm -f elf64 ex12.asm -o ex12.o`
- `gcc ex12.o -o ex12`
- `./ex12`

13 Exercice 13 – Implémenter `atoi`

Objectif

Convertir une chaîne de chiffres ASCII en entier (`int`).

Code source

- `ex13_atoi/ex13.asm`

Compilation et exécution

- `nasm -f elf64 ex13.asm -o ex13.o`
- `gcc ex13.o -o ex13`
- `./ex13`

14 Exercice 14 – Mini-calculatrice (+, -, *, /)

Objectif

Lire deux entiers et un opérateur, effectuer l'opération et afficher le résultat.

Code source

- `ex14_calc/ex14.asm`

Compilation et exécution

- `nasm -f elf64 ex14.asm -o ex14.o`
- `gcc ex14.o -o ex14`
- `./ex14`

15 Exercice 15 – Appeler une fonction ASM depuis C

Objectif

Écrire une fonction en assembleur appelée depuis un programme C, en respectant la convention d'appel System V.

Code source

- `ex15_from_c/ex15.asm`
- `ex15_from_c/main.c`

Compilation et exécution

- `nasm -f elf64 ex15.asm -o ex15.o`
- `gcc main.c ex15.o -o ex15`
- `./ex15`